

1. The biggest challenge in translating CloudFormation to Terraform was figuring out how to reference other resources. It's easier in CloudFormation because you just have to use commands like `!Sub` or `!Ref` to plug in values from other endpoints, which we did with our SNS endpoint. You have to be more explicit in Terraform, like telling it what resources need to be created first and also know the exact attribute name for each resource. One other challenge was connecting the S3 bucket in Terraform. In CloudFormation, it's intuitive because it keeps the S3 bucket notification config inside the resource, but in Terraform it's a separate resource that you have to manually connect to the bucket and SNS topic.
2. Lines 92-95 in `cloudformation.yaml`. `IpProtocol: tcp. FromPort: 8000. ToPort: 8000. CidrIp: "0.0.0.0/0"`. The element that grants the SNS subscription services permission is the security group inbound rule that opens port 8000 to all traffic. SNS delivers messages by making an HTTP post request to the notify endpoint on the EC2 instance, and if the port isn't open then the request is blocked.
3. When the CSV is loaded into S3, the SNS topic sends an HTTP post to notify the EC2 instance. The S3 key is then picked up by FastAPI and it runs anomaly detection. If notify returns an error, then SNS continues trying until it drops the message entirely. To make this production grade, I'd put an SQS queue between SNS and the EC2 instance so the messages are saved even when the instance is down.
4. The operations we used are `GetObject`, `PutObject`, `ListBucket`, and `GetBucketLocation`. You could replace the full access policy with just these four operations.

```
{ "Effect": "Allow", "Action": [ "s3:GetObject", "s3:PutObject", "s3:ListBucket",  
"s3:GetBucketLocation" ], "Resource": [ "arn:aws:s3:::YOUR-BUCKET-NAME",  
"arn:aws:s3:::YOUR-BUCKET-NAME/*" ] }
```

This removes `DeleteObject` and anything else that the app doesn't need.
5. If the system needed to process 100 times more files per hour, you'd need to run multiple EC2 instances, which could cause issues with the `baseline.json` file, because uploading files in two instances could lead to one overwriting the other. We can avoid this by adding an SQS queue between the SNS and EC2 to ensure that each file is being processed once. While a single instance is easier to implement, SQS is definitely more reliable.